

Problema A. Una Tada de Pizza Restante

Nombre del archivo Pizza.c, Pizza.cpp, Pizza.java, Pizza.py Estándar
fuente: Entrada: Estándar
Salida:

El departamento de CS acaba de tener una gran fiesta y pidió demasiada pizza. Ahora es hora de guardar las sobras. Se ordenaron varias pizzas pequeñas, medianas y grandes, y todavía quedan rebanadas en algunas o todas las cajas de pizza. Una pizza pequeña tiene 6 rebanadas, una mediana 8 y una grande 12. Para ahorrar espacio, puedes combinar las sobras de las mismas pizzas del mismo tamaño en una caja del tamaño correcto, pero no puedes colocar una rebanada en una caja de un tamaño diferente y no puedes llenar una caja con más rebanadas de las que originalmente contenía. ¿Cuál es el número mínimo de cajas que necesitarás para contener todas las sobras?

Entrada

La primera línea de la entrada contiene un entero positivo n ($n < 1000$), el número de pizzas pedidas. Cada una de las siguientes n líneas contiene dos elementos si y li (separados por un espacio) que representan las sobras para una pizza dada. si es una cadena S, M o L que representa el tamaño de la pizza i, y li es un entero que representa el número de rebanadas sobrantes para la pizza i. Puedes suponer que cada li está entre cero y el original número de rebanadas de ese tamaño de pizza, inclusive.

Salida

Imprime un solo número, el menor número total posible de cajas que puedan contener la pizza sobrante de acuerdo con las restricciones anteriores.

Ejemplo

Entrada	Salida
3 S 0 M 5 L 0	1
3 S 3 S 4 S 2	2
4 S 1 M 1 M 3 L 1	3
4 L 6 M 2 M 6 L 6	2

Problema B. Andor Ataca de Nuevo

Nombre del archivo: Andor.c, Andor.cpp, Andor.java, Andor.py Estándar
 fuente: Entrada: Estándar
 Salida:

Cassian Andor es un espía rebelde que se ha infiltrado en el principal arsenal de naves estelares del Imperio. Ha logrado hackear la computadora principal de la instalación, donde ha encontrado varios procesos de toma de decisiones para la próxima gran arma del Imperio: la Estrella de la Muerte (y vaya que queremos decir Muerte; el nombre aún está en desarrollo). Cada proceso de decisión tiene la forma de un árbol AND/OR, donde las hojas del árbol almacenan valores booleanos y los nodos internos son nodos AND o nodos OR, alternando a lo largo de cualquier camino desde la raíz. Un nodo AND evalúa como verdadero si todos sus subárboles son verdaderos y como falso en caso contrario; un nodo OR evalúa como verdadero si al menos uno de sus subárboles es verdadero y como falso en caso contrario. El valor de un árbol de decisión o cualquier subárbol es el valor que evalúa su raíz. Un ejemplo de un árbol AND/OR (que evalúa como verdadero) se muestra en la Figura 1.

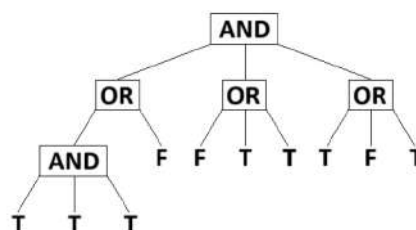


Figura 1. Ejemplo de árbol AND/OR

Cassian ha decidido sabotear cada árbol de decisión haciendo cambios en una o más hojas para invertir el valor devuelto por la raíz. Para que su sabotaje sea difícil de detectar, le gustaría cambiar el menor número posible de hojas. Por ejemplo, en el árbol anterior Cassian podría cambiar todas las hojas a falso para que el árbol evalúe como falso, pero puede lograr el mismo resultado cambiando solo una de las hojas verdaderas más a la izquierda a falso. Aunque Cassian es algo solitario, podría necesitar ayuda aquí. Ayúdale escribiendo un programa que, dado un árbol AND/OR, determine el número mínimo de hojas que deben cambiarse para

cambiar la evaluación del árbol.

Entrada

La entrada comienza con una línea que contiene dos valores n t , donde n ($2 \leq n \leq 20$) es el número de niveles en el árbol y t es A u O, indicando el tipo de nodos en los niveles impares del árbol; los nodos en los niveles pares serán del tipo opuesto. La raíz del árbol está en el nivel 1. A continuación hay n líneas que describen el árbol AND/OR. Cada línea contiene una o más entradas e_1 e_2 . . . e_m , donde cada e_i es el carácter T o F (indicando una hoja con valor verdadero o falso) o un entero positivo $v \leq 10$ que indica un nodo interno con v hijos. Hay un solo valor numérico en la primera de estas líneas; el número de entradas en cada línea siguiente es igual a la suma de los valores numéricos en el nivel anterior. Los nodos en cada nivel deben asignarse de izquierda a derecha a los nodos del nivel anterior. El número total de nodos en el árbol, tanto internos como hojas, es menor o igual a 105.

Salida

Imprime el número mínimo de hojas que deben cambiarse para invertir la evaluación del árbol.



Ejemplo

Entrada	Salida
4 A 3 2 3 3 3 F T F T F T T T T T	1
2 O 10 T T T T T T T T T T	10

Problema C. AROD

Nombre de archivo fuente: AROD.c, AROD.cpp, AROD.java, AROD.py

Entrada: Estándar

Salida: Estándar

Desde que se retiró de una exitosa carrera deportiva, Alex ha dedicado la mayor parte de su tiempo a analizar conceptos fundamentales de las matemáticas. Recientemente, se ha centrado en la categorización de triángulos según sus ángulos interiores, y ha inventado el acrónimo AROD para llevar un registro de los cuatro tipos fundamentales:

1. A = agudo: los tres ángulos son menores de 90 grados
2. R = recto: un ángulo es de 90 grados
3. O = obtuso: un ángulo es mayor que 90 grados, pero menor que 180 grados
4. D = degenerado: un ángulo es de 180 grados, o, equivalente, el tres vértices son colineales

Alex se pregunta cuántas veces tres vértices distintos elegidos de una rejilla regular de puntos x - y especifican un triángulo en cada una de las cuatro categorías AROD. Más precisamente, para enteros positivos m_x y m_y , quiere considerar todas las formas posibles de elegir tres vértices distintos del conjunto

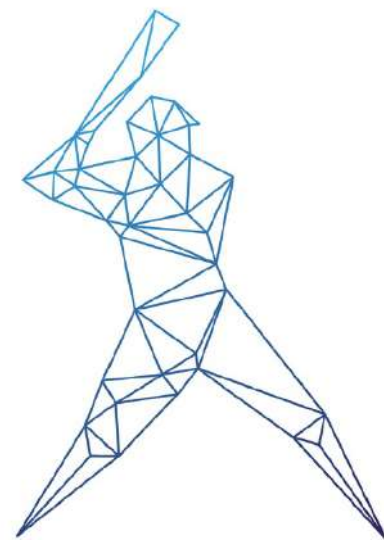


Imagen de Yakovliev (iStock); Usada bajo licencia

$$V(m_x, m_y) = \{(x, y) : x \text{ e } y \text{ son enteros, } 0 \leq x \leq m_x, 0 \leq y \leq m_y\}$$

y luego clasificar cada uno de los triángulos correspondientes en una de las cuatro categorías anteriores.

Entrada

La entrada es una línea que contiene dos enteros positivos, m_x y m_y , que satisfacen $m_x + m_y \leq 600$.

Salida

Imprimir cuatro líneas que contengan los números de veces que tres vértices distintos elegidos de $V(m_x, m_y)$ especifican un triángulo agudo, recto, obtuso o degenerado, en ese orden (un número por línea).

Ejemplo

Entrada	Salida
1 2	0 14 4 2
2 3	22 94 84 20

Problema D. Pero quiero ganar

Nombre del archivo fuente: Butiwant.c, Butiwant.cpp, Butiwant.java, Butiwant.py

Entrada: Estándar

Salida: Estándar

Borphee, la ciudad más grande de todo Frobozz, perdió recientemente a su alcalde debido al insaciable apetito de un Grue. El vicealcalde se convirtió en alcalde interino y asumió de inmediato las funciones de alcalde, que consisten principalmente en presidir los Campeonatos Anuales de Doble Fanucci (un juego de cartas extraordinariamente complejo) y el Festival de Canciones De Mal en Peor, donde los peores cantantes del reino se reúnen cada invierno. El salario del alcalde es de varios cientos de miles de zorkmids (zm) al año, lo cual es muy lucrativo para un trabajo tan fácil. Con la elección especial acercándose rápidamente, está claro que el vicealcalde quiere hacer todo lo posible para cambiar su estatus de "alcalde interino" a "alcalde" y conservar su generoso salario. Todo el imperio utiliza una variación de votación por orden de preferencia de un solo ganador para todas sus elecciones. Funciona así:

1. Los votantes marcan sus papeletas en orden de preferencia: primera opción (su candidato favorito), segunda opción (su siguiente favorito), y así sucesivamente. Cada votante debe clasificar a todos los candidatos en su papeleta. Por ejemplo, si hay 12 candidatos en la elección, cada votante clasificará sus opciones del 1 (su favorito) al 12 (el que menos le gusta). La elección procede entonces en una serie de rondas.
2. Al inicio de cada ronda, se cuentan todos los votos de primera opción. Si un candidato recibe más del 50% de los votos de primera opción, ese candidato gana y la elección termina.
3. Si no hay ganador, el candidato con el menor número de votos en primer lugar es eliminado de todas las papeletas, y los candidatos restantes suben en la clasificación. Por ejemplo, si un votante clasificó a cuatro candidatos en el orden A (primera opción) B C D y el candidato A es eliminado, la papeleta de este votante pasa a ser B (nueva primera opción) C D; si el candidato B es eliminado, la papeleta pasa a ser A C D. Si uno o más candidatos empatan con el menor número de votos en primer lugar, todos ellos son eliminados de las papeletas.
4. Después de reajustar las papeletas, vuelva al paso 2 para iniciar la siguiente ronda.

Este procedimiento puede continuar hasta que solo queden 2 candidatos, momento en el cual el candidato con la mayoría de votos en primer lugar gana. El alcalde interino asume que la elección finalmente se reducirá a él y a otro candidato muy popular como los dos principales contendientes. Con esto en mente, quiere concentrar sus esfuerzos de campaña en los votantes que clasificaron a otro candidato como su primera opción, en caso de que el conteo inicial de votos lo coloque en segundo lugar. Para ayudarlo a planificar mejor, quiere que desarrolles un programa que le permita determinar el número mínimo de rondas (después de la primera ronda) que le tomará obtener la mayoría de los votos, suponiendo que esté en segundo lugar después de la primera ronda.

Entrada

La entrada comienza con una línea que contiene un entero, C ($2 \leq C \leq 20$), que es el número de candidatos en la elección. Esto es seguido por una sola línea que contiene C enteros distintos $v_1, v_2, \dots, v_C$, donde cada v_i $1 \leq v_i \leq 109$ especifica el número de votos que recibió el candidato i .

Salida

La salida consiste en una sola línea. Si el candidato en segundo lugar no puede ganar la elección, entonces imprime IMPOSSIBLE TO WIN. De lo contrario, imprime el número mínimo de rondas (después de la primera ronda) que le tomará al candidato en segundo lugar ganar la elección usando votación por orden de preferencia.



Ejemplo

Entrada	Salida
3 1 70 67	IMPOSIBLE GANAR
6 1 2 13 12 70 69	3
3 11 2 10	1
3 1 2 3	IMPOSIBLE GANAR

Problema E. Solitario de Ajedrez

Nombre del archivo fuente: Chess.c, Chess.cpp, Chess.java, Chess.py
 Entrada: Estándar
 Salida: Estándar

El Solitario de Ajedrez es una versión del ajedrez diseñada para ser jugada por una sola persona. Bien, probablemente ya lo habías deducido, pero las reglas exactas son las siguientes: dado un tablero con 2 o más piezas de ajedrez, ninguna de las cuales es un peón, encuentra una serie de capturas que resulten en una sola pieza en el tablero. Cualquier pieza que se mueva debe capturar otra pieza, así que si inicialmente hay m piezas en el tablero, la solución implica $m - 1$ movimientos. Un ejemplo de rompecabezas y su solución se muestra en la Figura 2 (que corresponde al Ejemplo de Entrada 1):

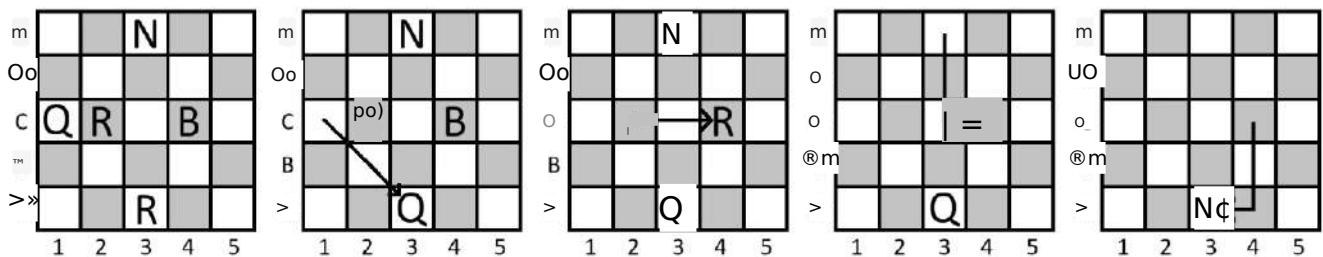
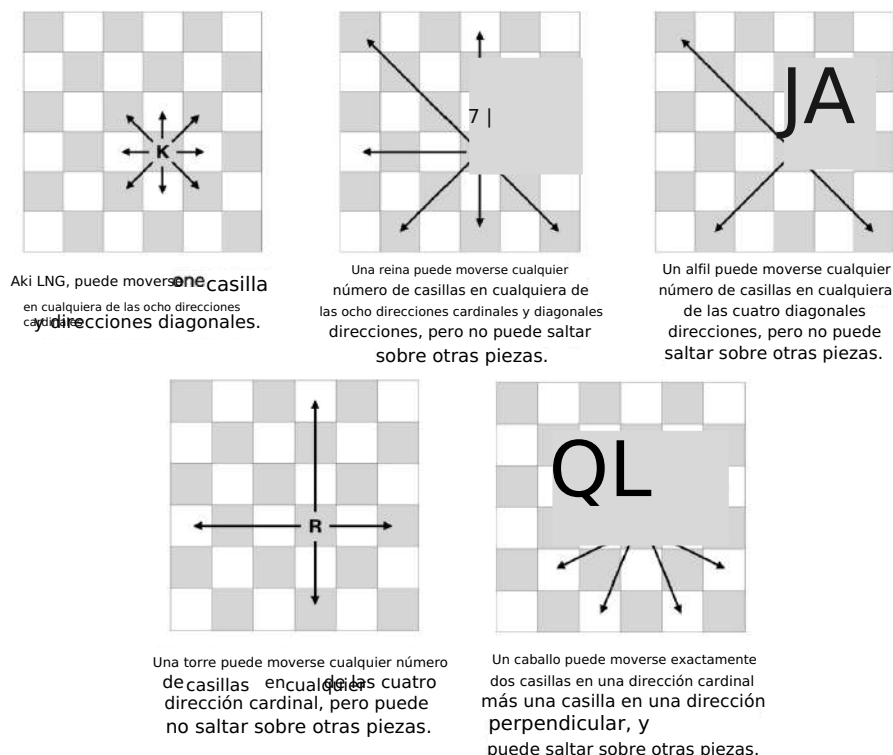


Figura 2. Problema de solitario de ajedrez y una solución

Como recordatorio, aquí se muestra cómo las diferentes piezas de ajedrez pueden capturar a otras piezas:



Para este problema, se te dará un rompecabezas de solitario de ajedrez y deberás dar una serie de movimientos para resolverlo.

Entrada

La entrada comienza con una línea $n\ m$ donde n ($2 \leq n \leq 8$) es el número de filas y columnas del tablero, y m ($2 \leq m \leq 10$) es el número de piezas en el problema. A continuación hay m líneas de la forma $p\ loc$,

donde p es uno de 'N', 'B', 'R', 'Q', 'K' indicando respectivamente caballo, alfil, torre, reina y rey, y loc es una cadena de longitud 2 que indica la ubicación de la pieza en el tablero. El primer carácter es una letra mayúscula que indica la fila y el segundo carácter es un número entero que indica la columna (como se muestra en la Figura 2). Todas las ubicaciones son válidas y no habrá dos ubicaciones iguales.

Salida

Imprime m−1 líneas mostrando los m−1 movimientos para resolver el problema. Cada línea será de la forma p: loc1 -> loc2 indicando un movimiento de la pieza p desde la ubicación loc1 a la ubicación loc2. Si hay múltiples soluciones, si imprime la que tenga el primer movimiento con el loc1 lexicográficamente menor. Si aún hay empate, imprime la que tenga el primer movimiento con el loc2 lexicográficamente menor. Si aún hay empate, aplica estas reglas al segundo movimiento, y así sucesivamente.

Si no hay solución posible al problema, imprime No solution

Ejemplo

Entrada	Salida
5 5 N E3 Q C1 R C2 B C4 R A3	Q: C1 -> A3 R: C2 -> C4 N: E3 -> C4 N: C4 -> A3
4 4 Q D4 Q B1 Q C2 K A3	No solution



Problema F. Secuencia Fraccional

Nombre del archivo fuente: Sequence.c, Sequence.cpp, Sequence.java, Sequence.py
Entrada: Estándar
Salida: Estándar

Considere la siguiente secuencia creciente, S , de números racionales:

$$1, 2, 2\frac{1}{2}, 3, 3\frac{1}{3}, 3\frac{2}{3}, 4, 4\frac{1}{4}, 4\frac{1}{2}, 4\frac{3}{4}, 5, 5\frac{1}{5}, 5\frac{2}{5}, 5\frac{3}{5}, 5\frac{4}{5}, 6, \dots$$

S se compone de un conjunto infinito de bloques, $N_1, N_2, N_3, \dots$, donde el bloque N_i es

$$i, i + 1/i, i + 2/i, \dots, i + (i - 1)/i.$$

Así $S(1) = 1$, $S(2) = 2$, $S(3) = 2\frac{1}{2}$, etc. Escribe un programa que tome como entrada un entero n y devuelva $S(n)$.

Entrada

La entrada es una sola línea que contiene un entero, n ($1 \leq n \leq 4 \cdot 10^9$).

Salida

Salida $S(n)$ como un único entero si la respuesta es un número entero. De lo contrario, muestra la parte entera, un espacio y una fracción propia a/b en su forma más reducida ($0 < a < b$ y $\text{GCD}(a, b) = 1$). Véase el ejemplo (p. ej., salidas de ejemplo).

Ejemplo

Entrada	Salida
326	26
448	30 2/5
4000000000	89443 19596/89443

Problema G. ¿Cuántas pelotas?

Nombre de archivo fuente: Balls.c, Balls.cpp, Balls.java, Balls.py
Entrada: Estándar
Salida: Estándar

Si una bolsa contiene r pelotas rojas y g pelotas verdes y se extraen dos pelotas al azar, la probabilidad de obtener una pelota de cada color es

$$P(r, g) = \frac{2rg}{(r+g)(r+g-1)}$$

Escribe un programa que tome como entrada un número racional p/q en términos más reducidos y determine si existe un número $r \leq 10^6$ y un $g \geq r$ para el cual $P(r, g) = p/q$.

Entrada

La única línea de entrada contiene dos enteros positivos separados por espacios p ($p > 0$) y q ($2p-1 \leq q \leq 1000$). Se garantiza que estos dos enteros son primos entre sí.

Salida

Si hay una solución, imprime los dos enteros positivos r y g que satisfacen las condiciones anteriores, separados por un espacio. Si hay varias soluciones, imprime la que tenga el menor valor de r . Para cualquier valor de r , existe a lo sumo un valor de g ($g \geq r$) que resuelva $P(r, g) = p/q$. palabra imposible.

Si no hay solución con $r \leq 10^6$, imprime el P(r, g) = p/q. palabra imposible.

Ejemplo

Entrada	Salida
12 25	9 16
8 25	imposible

Problema H. ¡Muévete, Lento!

Nombre del archivo Slowpoke.c, Slowpoke.cpp, Slowpoke.java, Slowpoke.py Estándar
fuente: Entrada:
Salida: Estándar

Centerville tiene un pequeño problema. Debido a recientes obras viales, muchos vehículos lentos (camiones de basura, camiones de reparto, etc.) han sido desviados por la ciudad. Muchos ciudadanos (o al menos quien escribe este problema) están cada vez más molestos por quedarse atrapados detrás de estos vehículos, especialmente cuando permanecen en una misma carretera continua durante un período prolongado. El consejo municipal ha promulgado recientemente una ordenanza que establece que ningún vehículo lento puede permanecer en dos o más carreteras

consideradas "continuas" más allá de una distancia d . Los detalles de la ordenanza son los siguientes:

1. La ordenanza comienza con una definición de lo que es un "vehículo lento". Eso no es importante para nosotros aquí (aunque sabes reconocer uno cuando lo ves).
2. La ordenanza luego enumera los pares ordenados de carreteras en la ciudad que se consideran "continuas". Los vehículos lentos pueden circular por cualquiera de las carreteras de un par por separado (incluso si la longitud de cualquiera de ellas es $> d$), pero no pueden circular por la primera carretera seguida inmediatamente de la segunda si la longitud total del par es $> d$. Ten en cuenta que si la segunda carretera de un par continuo coincide con la primera carretera de otro par continuo, entonces las tres carreteras de los dos pares se consideran juntas continuas. En otras palabras, si (por ejemplo) $A \rightarrow B \rightarrow C$ se considera continua y $B \rightarrow C \rightarrow D$ se considera continua, entonces $A \rightarrow B \rightarrow C \rightarrow D$ se considera continua.
3. La ordenanza termina con métodos para determinar d (de nuevo, estos detalles no son importantes para nosotros aquí).

Obviamente, los dueños de estos vehículos lentos están algo molestos (bien merecido, diría yo, pero de nuevo eso no es importante para nosotros). Mientras que encontrar el camino más corto entre dos puntos solía ser sencillo (sin doble sentido), ahora se ha vuelto un reto debido a las restricciones impuestas; de hecho, puede que ni siquiera sea posible ir de un punto a otro. Por ejemplo, considera el conjunto de carreteras en la Figura 3. Supón que un camión lento debe ir desde la intersección A hasta la intersección D, y que

tres pares de carreteras se consideran continuas: $A \rightarrow B \rightarrow C$; $A \rightarrow B \rightarrow E$; y $B \rightarrow F \rightarrow G$. Si d es 30 (o mayor), entonces el camión puede ir por $A \rightarrow B \rightarrow C \rightarrow D$. Si d es 25, esta ruta ya no es utilizable, así que la ruta más corta ahora es $A \rightarrow B \rightarrow E \rightarrow C \rightarrow D$ (Ejemplo de Entrada 1). Si d es 15, entonces $A \rightarrow B \rightarrow E$ ya no es utilizable, así que la ruta más corta ahora es $A \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow D$ (nota que en este caso se permite ir de A a B aunque la longitud de esa carretera sea $> d$). Finalmente, si d es < 14 , entonces es imposible que este camión vaya de A a D (Ejemplo de Entrada 2). Ten en cuenta que los vehículos lentos no pueden hacer giros en U sin causar aún más retrasos, así que, por ejemplo, el camino $A \rightarrow B \rightarrow F \rightarrow B \rightarrow C \rightarrow D$ no es posible para ningún valor de d . Dada una especificación de la red vial, dos intersecciones s y t , y un valor para d , los dueños de los vehículos lentos quieren saber la distancia más corta para viajar desde la intersección s hasta la intersección t .

Entrada

La entrada comienza con una línea que contiene 6 enteros n m k d s t , donde n ($2 \leq n \leq 100$) es el número de intersecciones en la ciudad (numeradas del 1 al n), m es el número de carreteras entre intersecciones, k ($0 \leq k \leq m(m-1)$) es el número de pares ordenados de carreteras que se consideran continuas cuando se recorren consecutivamente, d ($1 \leq d \leq 100$) es la distancia máxima que los vehículos lentos pueden recorrer en una secuencia continua de carreteras, s ($1 \leq s \leq n$) es la intersección desde la que se viaja, y t ($1 \leq t \leq n$, $s \neq t$) es la intersección a

la que se viaja. A continuación hay m líneas, cada una con tres enteros a , b , l ($1 \leq a, b \leq n$, $a \neq b$, $1 \leq l \leq 100$) indicando que una carretera de doble sentido de longitud l conecta las intersecciones a y b . Hay como máximo una carretera entre

cualquier dos intersecciones. A continuación hay k líneas, del tipo $a\ b\ c$ ($a \neq b, b \neq c, a \neq c$) que indican que conducir por la carretera de a a b seguida de la carretera de b a c se considera conducción continua. Se garantiza que hay una carretera entre a y b y que hay una carretera entre b y c . Tenga en cuenta que esto no significa que el orden inverso de conducir por esas carreteras se considere conducción continua.

Salida

Si es imposible viajar desde s para t salida imposible; de lo contrario, indique la distancia más corta a viajar desde s hasta t .

Ejemplo

Entrada	Salida
7 8 3 25 1 7 1 2 20 2 3 10 2 4 4 4 3 8 2 5 6 5 6 8 6 3 4 3 7 10 1 2 3 1 2 4 2 5 6	42
7 8 3 12 1 7 1 2 20 2 3 10 2 4 4 4 3 8 2 5 6 5 6 8 6 3 4 3 7 10 1 2 3 1 2 4 2 5 6	imposible

Explicación

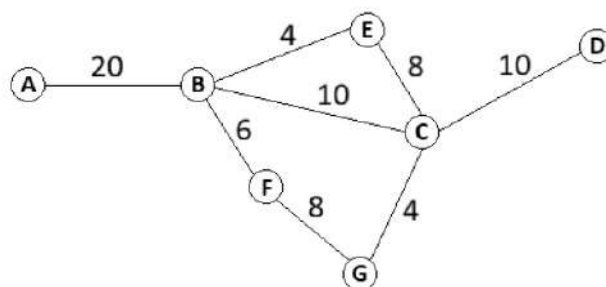


Figura 3. Red de carreteras de ejemplo, utilizada por las Entradas de Ejemplo 1 y 2.

Problema I. Pirámide de Números

Nombre del archivo: Pyramid.c, Pyramid.cpp, Pyramid.java, Pyramid.py
 Estándar: Estándar
 Salida: Estándar

Unedtoth, Inc. es una empresa que crea juegos matemáticos para estudiantes de primaria, y Pam D. Monium acaba de ser puesta a cargo de su último producto: Pirámides de Números. El juego consiste en tarjetas, cada una con una pirámide de cuadros, algunos llenos con números y otros vacíos (ver la primera tarjeta a la izquierda en la Figura 4). El objetivo del juego es encontrar valores para colocar en cada cuadro vacío de modo que el valor en cada cuadro (excepto los de la base) sea la suma de los valores en los dos cuadros debajo de él. Para mantenerlo, bueno, elemental, los valores finales en los cuadros siempre son enteros con valor absoluto < 100 . Las tarjetas también están diseñadas para que puedan resolverse en varias pasadas, cada pasada de arriba hacia abajo y de izquierda a derecha en cada fila, llenando cualquier valor que pueda determinarse por sus vecinos. Un ejemplo de esto se muestra en las tres tarjetas restantes de la Figura 4. Tenga en cuenta que puede haber otras formas de resolver una tarjeta dada, pero si no se puede encontrar una solución única usando este método, la tarjeta no debe entregarse a los estudiantes. Pam llama a estas tarjetas "ambiguas".

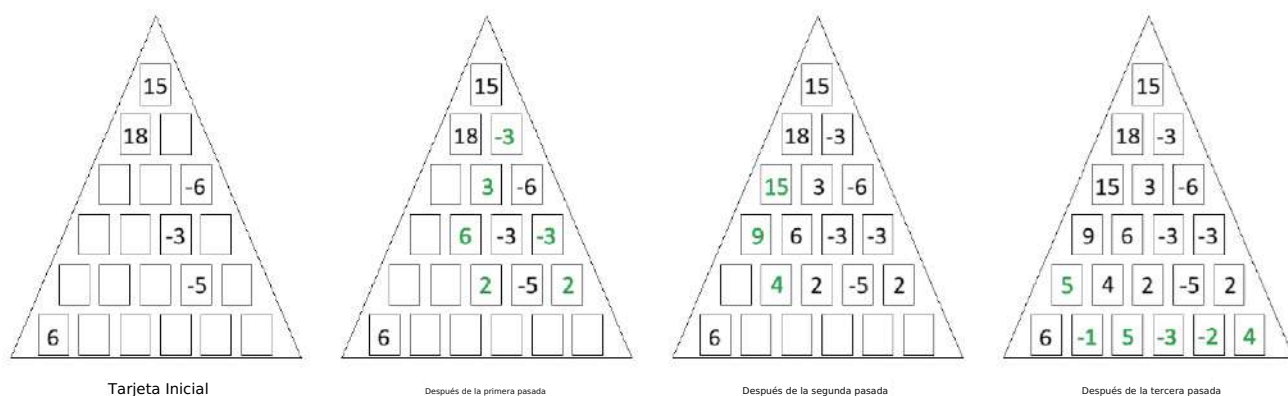


Figura 4. Ejemplo de tarjeta de Pirámide de Números y una forma de resolverla.

Todo parecía ir bien para Pam, con las máquinas en la fábrica de Unedtoth produciendo tarjeta tras tarjeta. Pero cuando Pam comenzó a hacer pruebas de producto, notó que había un fallo en la máquina y algunas tarjetas eran ambiguas o llevaban a valores contradictorios en uno o más cuadros. Por ejemplo, si a la tarjeta de la izquierda en la Figura 4 le faltara el 6 en la esquina inferior izquierda, esa tarjeta sería ambigua porque permitiría múltiples soluciones; si esa tarjeta tuviera un 5 en el cuadro más a la derecha de la última fila, o un 3 en el cuadro más a la derecha de la segunda fila desde arriba, cada uno de estos casos llevaría a una contradicción y la tarjeta no tendría solución. Finalmente, si el 18 se cambiara por un -90, no habría solución ya que uno de los valores resultantes estaría fuera del rango $-99 \leq v \leq 99$. Según Pam, hay dos caminos posibles ahora. El primero es ignorar las pruebas y enviar las tarjetas, confiando en las malas habilidades matemáticas de los niños estadounidenses para ganar tiempo y buscar otro trabajo; o puede pedir a alguien que escriba un software para detectar y eliminar las tarjetas defectuosas. Tras mucha reflexión y revisar su cuenta bancaria, ha decidido lo segundo y ha acudido a ti en busca de ayuda.

Entrada

La entrada comienza con una línea n ($2 \leq n \leq 100$) que es el número de filas en una tarjeta de pirámide. A continuación hay n líneas que especifican las filas de la pirámide. La primera línea contiene un solo valor, la segunda línea dos valores, y así sucesivamente. Todos los valores v serán enteros en el rango $-99 \leq v \leq 100$, donde el valor especial 100 indica un cuadro vacío.

Salida

Salida una de tres cosas: si la ficha tiene múltiples soluciones con todos los valores en el rango -99, ..., 99, salida ambigua; si la ficha no tiene solución, o tiene una solución con uno o más valores fuera del rango -99,..., 99, salida sin solución; de lo contrario, salida solvable y luego la solución usando el formato mostrado en la Salida de Ejemplo.

Ejemplo

Entrada	Salida
6 15 18 100 100 100 -6 100 100 -3 100 100 100 100 -5 100 6 100 100 100 100 100	resoluble 15 18 -3 15 3 -6 9 6 -3 -3 5 4 2 -5 2 6 -1 5 -3 -2 4
2 100 10 100	ambiguo
2 10 10 10	sin solución
2 100 51 51	sin solución

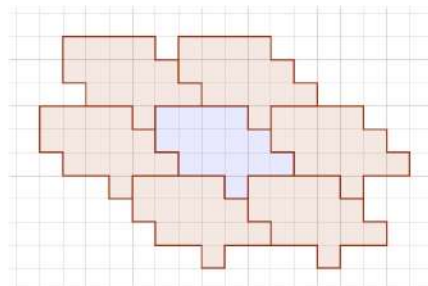
Problema J. Mosaico de polinomio

Nombre del archivo fuente: Polyomino.c, Polyomino.cpp, Polyomino.java, Polyomino.py

Entrada: Estándar

Salida: Estándar

Un poliomino es una unión conexa no vacía de cuadrados unitarios sin agujeros, donde las esquinas de cada cuadrado unitario están en coordenadas enteras en el plano bidimensional. Un polinomio puede describirse mediante una representación en cadena de su contorno. Empezando en ciertas coordenadas enteras en el contorno del polinomio, podemos seguir el contorno dando pasos unitarios hacia arriba, derecha, abajo o izquierda. Representamos cada uno de estos pasos con las letras u, r, d y l, respectivamente. Si concatenamos estas letras mientras seguimos el contorno del polinomio hasta alcanzar las coordenadas de inicio, entonces la concatenación se llama la cadena de contorno para el polinomio. Tenga en cuenta que mientras se sigue el contorno,



Parte de una teselación para el primer polinomio en el entrada de ejemplo.

solo las coordenadas iniciales pueden visitarse dos veces; todas las demás coordenadas en la ruta deben visitarse exactamente una vez.

Las rotaciones cíclicas de la cadena de contorno se obtienen iniciando en diferentes coordenadas en el contorno y siguiendo en el mismo orden que en la cadena de contorno (ya sea en sentido horario o antihorario, pero no ambos). Por ejemplo, las rotaciones cíclicas de urdl son urdl, rdul, dlur y lurdl.

Para una cadena S , que consiste en las letras u, r, d y l, define $\bar{S}$ como la cadena obtenida invirtiendo las letras de S y cambiando cada u por d, cada d por u, cada r por l y cada l por r. Por ejemplo, para $S = \text{ururrdl}$, tenemos $\bar{S} = \text{rulddld}$.

Se puede demostrar que un polinomio tilde el plano por traslaciones (es decir, sin rotaciones ni reflexiones) si y solo si una rotación cíclica de la cadena de contorno B puede escribirse como $\bar{B} = \bar{X} \cdot \bar{Y} \cdot Z \cdot X \cdot Y \cdot Z$ o $B = X \cdot Y \cdot X \cdot \bar{Y}$, donde X, Y, Z son cadenas no vacías. Con frecuencia, puede haber muchas formas de rotar la cadena de contorno y escribirla en una o ambas de estas dos formas.

Dada una cadena de contorno para un polinomio, cuenta el número de formas en que cada rotación cíclica B de la cadena de contorno puede escribirse como $B = \bar{X} \cdot \bar{Y} \cdot X \cdot Y$ o $B = X \cdot Y \cdot \bar{Z} \cdot \bar{X} \cdot \bar{Y} \cdot Z$, donde X, Y y Z son cadenas no vacías. Este recuento será 0 si el polinomio no puede tilde el plano por traslaciones.

Entrada

La entrada consiste en una única línea que contiene dos valores k y s , donde k ($4 \leq k \leq 10000$) es la longitud de la cadena de contorno y s es la cadena de contorno. La cadena de contorno consistirá en las letras u, r, d y l.

Salida

Imprime un solo entero, el número de formas en que cada rotación cíclica de la cadena de contorno puede escribirse en la forma $X \cdot Y \cdot X \cdot \bar{Y}$ o $\bar{X} \cdot \bar{Y} \cdot Z \cdot X \cdot Y \cdot Z$.

Ejemplo

Entrada	Salida
20 uurrdrdrdrldlullul	6
14 ururdrdrldlul	4
12 uurdrurddl	0
8 urrrdl	16

Problema K. Tríptico

Nombre de archivo fuente: Triptych.c, Triptych.cpp, Triptych.java, Triptych.py

Entrada: Estándar

Salida: Estándar

Donald es el curador de un museo especializado en artefactos medievales. Está especialmente orgulloso de un tríptico recientemente adquirido (una obra de arte de tres paneles) que está llena de iconografía religiosa. Quiere convertir este tríptico en la atracción principal de la próxima exposición del museo y planea destacar uno de los tres paneles durante cada semana de la exposición iluminándolo con un foco especial.



Imagen de la National Gallery of Art; Pública Dominio

Como pasante en el departamento de TI del museo, tu tarea principal es programar el microcontrolador que opera el foco. Para simplicidad, los tres paneles están etiquetados como A, B, C, de izquierda a derecha, y la entrada al microcontrolador será una cadena de estos caracteres, que especifica, en orden, el panel que debe destacarse durante cada semana de la exposición (la cual puede durar hasta un año). Dicha cadena se llama secuencia de iluminación.

Además de programar el microcontrolador, Donald te ha pedido generar algunas posibles secuencias de iluminación, de las cuales él elegirá una. Sin embargo, no todas las secuencias de caracteres de A, B, C son aceptables, ya que Donald es muy estricto con las siguientes reglas:

1. **variedad:** no se puede resaltar el mismo panel durante dos semanas consecutivas, excepto el panel central (B), que está diseñado para ser (literalmente) el centro de atención; el panel central puede resaltarse durante dos semanas consecutivas, pero no durante tres o más semanas consecutivas.
2. **equilibrio:** Ningún panel debe resaltarse significativamente más que cualquier otro panel. Más precisamente, Donald ha elegido un entero no negativo, d , y debes asegurar que cualquier par de a, b, c difiera como mucho en d , donde a, b y c son los números de ocurrencias de A, B y C, respectivamente, en cualquier secuencia de iluminación.
3. **aibohphobia** (miedo a los palíndromos) - Donald tiene una aversión extrema a los palíndromos (nadie se atreve a preguntar por qué), así que ninguna secuencia de iluminación puede ser un palíndromo (igual hacia adelante y hacia atrás).

Dado el número de semanas de la exposición y el valor de d , determina el número total de secuencias de iluminación que puedes generar para que Donald elija.

Entrada

La entrada consiste en una línea que contiene dos enteros, w ($2 \leq w \leq 52$) y d ($0 \leq d \leq 10$), el número de semanas de la exposición y el valor de la "diferencia máxima" descrita arriba, respectivamente.

Salida

Imprime un único entero: el número total de secuencias de iluminación de longitud w que puedes generar que cumplen todas las reglas de Donald.

Ejemplo

Entrada	Salida
3 0	6
2 0	0
4 1	24

Problema L. Gaviotas del Valle

Nombre de archivo fuente: Valley.c, Valley.cpp, Valley.java, Valley.py

Entrada: Estándar

Salida: Estándar

El complejo Cliff Valley se llama así por los acantilados empinados y verticales que encierran sus series de valles y colinas. Además de esta disposición geográfica única, una de las cosas por las que más es conocido son las Gaviotas del Valle de Grietola Azul de garganta azul, que anidan en las colinas escénicas del complejo. Otra cosa por la que es conocido es la lluvia; mucha lluvia. Esto puede hacer que sea difícil para las Gaviotas del Valle de Grietola Azul de garganta azul, ya que a menudo sus nidos deben abandonarse debido a las aguas de inundación que suben. Famosas por su belleza única y su canto de ave inquietante, la especie no es la más brillante a la hora de elegir hábitats de nidación. La ambientalista Audrey Bond ha sido puesta a cargo de modelar las inundaciones en el valle y la amenaza que esto supone para las gaviotas. Ha decidido utilizar simples cortes 2-D del valle para simplificar las cosas. Ella tiene información algo limitada sobre la disposición del valle y solo tiene un pequeño conjunto de puntos de elevación de una encuesta anterior con la que trabajar, los primeros y últimos de los cuales se encuentran en las paredes del acantilado (ver Figura 5a). También sabe que no hay nidos en estos puntos, ya que la Gaviota de garganta azul puede ponerse bastante rebelde cuando un topógrafo se acerca a su nido. Dados estos puntos, Audrey tiene la intención de ajustar una curva de interpolación por tramos, de primer grado, para modelar cada corte del valle. En otras palabras, va a trazar líneas rectas entre cada par de puntos consecutivos. Se muestra un ejemplo en la Figura 5b.

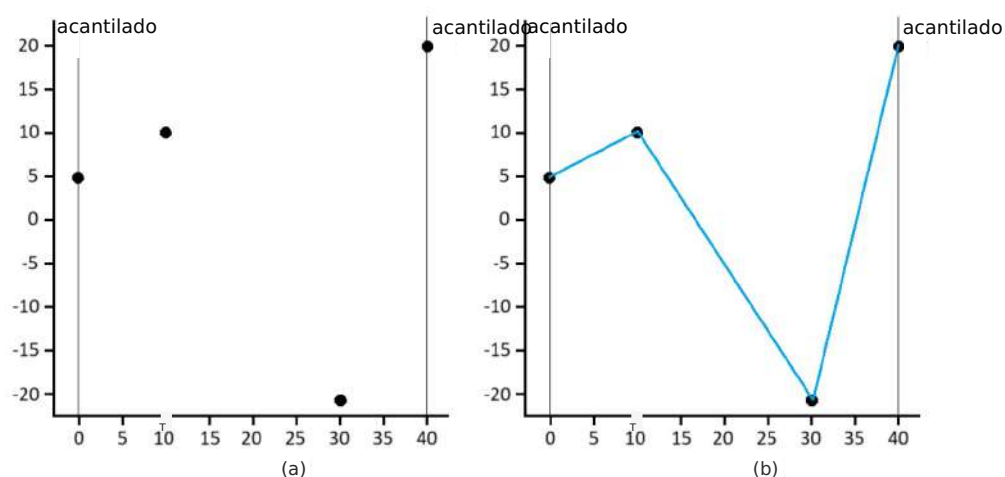


Figura 5. (a) Puntos de encuesta (Entrada de ejemplo 1). (b) Conexión por línea recta de puntos

Para una tasa de lluvia dada, Audrey calcula que será fácil determinar cuánto tardarán las aguas de la inundación en alcanzar cualquier nido de la Gaviota de garganta azul en el valle. O al menos ella pensó que sería fácil; resulta que incluso modelar con líneas rectas es un poco más difícil de lo que pensaba. ¿Puedes ayudar a una gaviota?

Entrada

La primera línea de la entrada contiene tres valores p r m donde p ($2 \leq p \leq 50$) es un entero positivo que indica el número de puntos de encuesta, r ($0.0 < r \leq 3.0$) es la tasa de lluvia en pies por hora, y m ($1 \leq m \leq 100$) es un entero positivo que indica el número de nidos. El valor de r tendrá como máximo dos dígitos después del punto decimal. A continuación hay una única línea con p pares de valores enteros x_i y_i ($0 \leq x_i \leq 5000$, $-1000 \leq y_i \leq 1000$) que especifican los p puntos de encuesta, en pies. Ningún par de puntos de encuesta tendrá el mismo valor de x o de y . Los valores de x están en orden ascendente y no tres puntos consecutivos serán colineales. Los primeros y últimos valores de x corresponden a las ubicaciones del acantilado y se puede suponer que las alturas de los lados del acantilado son mayores.

que todas las y de los puntos de encuesta. La línea final de la entrada contiene m enteros n_1 n_2 . . . n_m

($x_1 < n_1 < n_2 < \dots < n_m < x_p$) que indica los valores de x de las m localizaciones de nidos. Ningún valor y correspondiente a estos x es igual a ningún valor y de un punto de la encuesta.

Salida

Imprimir m líneas, una para cada una de las ubicaciones de nido correspondientes, especificando el tiempo en horas en que las aguas de la inundación llegarían al nido, asumiendo que la lluvia comienza en $t = 0$ y continúa a una tasa constante de r . Las respuestas se juzgarán correctas si tienen un error absoluto o relativo no mayor a 10^{-5} .

Ejemplo

Entrada	Salida
4 0.2 2	4.50000000
0 5 10 10 30 -20 40 20	21.68750000
6 18	